

Construire un projet Titanic ML de bout en bout

Tout ce qui a été fait, dans l'ordre, avec les commandes exactes et les vérifications effectuées à chaque étape — rédigé pour quelqu'un qui découvre comment monter un projet de data science propre, versionné et testé.

Projet : `titanic_ml` (pipeline scikit-learn + API FastAPI + dashboard Streamlit)

Dépôt : github.com/BonnieNac/agentik_ML_Titanic

Outils : uv, ruff, mypy, pytest, git, GitHub CLI, Altair, Playwright

Document généré le 22 septembre 2026

Sommaire

Point de départ : de quoi part-on ?

Le squelette de projet déjà en place

Les outils qui encadrent le travail de l'IA (dossier `.claude/`)

Étape 1 — Documenter le projet pour les futures sessions (`CLAUDE.md`)

Étape 2 — Mettre le projet sous Git et le pousser sur GitHub

Diagnostiquer l'état du dépôt

Commiter avec les *conventional commits*

Le piège HTTPS vs SSH

Étape 3 — Récupérer un projet tiers (fork avec `gh`)

Étape 4 — Concevoir des visualisations de données propres

La méthode avant le code : choisir la forme, puis la couleur

Explorer l'existant avant d'ajouter du code

Écrire le module de visualisations

Brancher le nouvel onglet dans l'application

Étape 5 — Le filet de sécurité qualité : lint, typage, tests

Étape 6 — Faire tourner l'application pour de vrai (pas juste la relire)

Premier niveau de preuve : rendre les graphiques en image

Deuxième niveau de preuve : un vrai navigateur headless

Un obstacle système résolu sans droits root

Étape 7 — Livrer la fonctionnalité (commit, confirmation, push)

Étape 8 — Un vrai bug remonté par l'utilisatrice : diagnostic

Étape 9 — Corriger le bug pour de bon (bissection de versions)

Étape 10 — Clôturer proprement

Ce qu'il faut retenir : les principes transférables à tout projet

1. Point de départ : de quoi part-on ?

1.1 Le squelette de projet déjà en place

Le projet `titanic_ml` n'est pas parti de zéro : il utilise un squelette ("cookiecutter") pensé pour la data science, qui pose dès le départ une structure standard :

- `src/titanic_ml/` — le code du package Python installable (pipeline ML, API)
- `app/` — l'application Streamlit (le tableau de bord visuel)
- `tests/` — les tests automatisés
- `data/`, `models/`, `docs/figures/` — données et artefacts générés, ignorés par Git
- `pyproject.toml` — la carte d'identité du projet : dépendances, configuration de `ruff`, `mypy`, `pytest`
- `Makefile` — des raccourcis (`make train`, `make test`, `make run` ...)

Pourquoi c'est important à savoir avant de coder : un des premiers réflexes à avoir sur un projet inconnu n'est pas d'écrire du code, mais de *lire* — le `README.md`, le `pyproject.toml`, le `Makefile` — pour comprendre les conventions déjà en place plutôt que d'en réinventer d'autres qui entreraient en conflit.

1.2 Les outils qui encadrent le travail de l'IA (dossier `.claude/`)

Ce projet contient un dossier `.claude/` qui configure la manière dont l'assistant Claude Code peut agir dessus. Concrètement, un fichier `.claude/settings.json` définit des règles :

Catégorie	Effet
<code>allow</code>	Commandes exécutées sans demander confirmation (<code>uv sync</code> , <code>uv run</code> , <code>pytest</code> , <code>git status</code> ...)
<code>ask</code>	Toujours demander confirmation (<code>git rebase</code> , <code>git reset</code>)
<code>deny</code>	Interdit purement et simplement (<code>rm -rf</code> , <code>git push</code> , <code>pip install</code> , lecture de <code>.env</code>)

Un *hook* (script déclenché automatiquement) lance `ruff check --fix` et `ruff format` après chaque fichier modifié, pour que le code reste toujours propre sans y penser.

Pourquoi c'est un bon réflexe à reproduire : même sans assistant IA, définir des garde-fous explicites (quelles commandes sont sûres, lesquelles demandent réflexion, lesquelles sont interdites) évite les accidents — que ce soit pour une IA ou pour un·e nouvel·le arrivant·e sur l'équipe.

2. Étape 1 — Documenter le projet (CLAUDE.md)

La toute première action a été de lancer la commande `/init`, qui demande à l'assistant d'analyser tout le dépôt et de produire (ou mettre à jour) un fichier `CLAUDE.md` : une sorte de "mode d'emploi" du projet à l'intention des futures sessions de travail.

Concrètement, cela a impliqué de lire, dans l'ordre :

1. `README.md`, `ARCHITECTURE.md`, `CONTRIBUTING.md`
2. `pyproject.toml` (dépendances, configuration des outils qualité)
3. `Makefile` (les commandes "officielles" du projet)
4. Le code source (`src/titanic_ml/core/...`, `src/titanic_ml/api/...`) pour comprendre l'architecture réelle
5. Le dossier `.claude/` et son propre `README.md` explicatif

Un `CLAUDE.md` existait déjà et était déjà très complet (commandes, architecture, pièges de style, conventions de test). Plutôt que de le réécrire, une seule section manquante a été ajoutée : **"Claude Code environment in this repo"**, qui résume les permissions `deny / ask`, le hook de formatage automatique, et les outils (sous-agent de revue, skill de description de PR) disponibles dans `.claude/`.

Pourquoi documenter l'architecture plutôt que de lister les fichiers :

un bon document d'onboarding explique le *pourquoi* des choix structurants (ex. "le cœur ML est indépendant de l'API et du dashboard") plutôt que de recopier un arbre de fichiers, qui devient vite obsolète et n'apprend rien qu'un simple `ls` n'apprendrait déjà.

3. Étape 2 — Mettre le projet sous Git et le pousser sur GitHub

3.1 Diagnostiquer l'état du dépôt

Avant tout push, on vérifie toujours l'état réel du dépôt local :

```
git remote -v
git status
git log --oneline -15
git branch -a
```

Constat : le dépôt était déjà initialisé, avec 7 commits d'historique, et un `remote origin` pointant déjà vers `https://github.com/BonnieNac/agentML_Titanic.git` — mais ce dépôt GitHub était vide (aucune branche poussée), confirmé avec :

```
git fetch origin
git ls-remote origin # ne renvoie rien -> dépôt distant vide
```

3.2 Committer avec les *conventional commits*

La modification du `CLAUDE.md` (étape précédente) était encore en attente (`git status` montrait `modified: CLAUDE.md`). Elle a été committée en respectant le format **conventional commits**, déjà utilisé dans l'historique du projet :

```
git add CLAUDE.md
git commit -m "docs: documente les garde-fous et outils Claude Code dans CLAUDE.md"
```

Qu'est-ce que les "conventional commits" ? C'est une convention de rédaction des messages de commit sous la forme `type: description`, où `type` est un mot-clé standard :

Type	Usage
<code>feat</code>	Nouvelle fonctionnalité (ex. le tableau de bord de visualisations)
<code>fix</code>	Correction de bug

docs	Documentation uniquement
refactor	Réorganisation du code sans changer son comportement
style	Formatage pur (espaces, indentation)
chore	Tâche de maintenance (config, dépendances)

L'intérêt : un historique lisible d'un coup d'œil, et la possibilité d'automatiser des choses (changelog, numéro de version) à partir des messages de commit — ce projet utilise d'ailleurs `commitizen` (`uv run cz bump`) pour ça.

3.3 Le piège HTTPS vs SSH

Premier essai de push, avec le remote existant (en HTTPS) :

```
git push -u origin main
```

Erreur rencontrée : `fatal: could not read Username for 'https://github.com': No such device or address` — Git essaie de demander un identifiant de manière interactive, ce qui est impossible dans un terminal non-interactif piloté par un script.

Diagnostic : vérifier si une clé SSH est disponible et déjà reconnue par GitHub :

```
ls -la ~/.ssh
ssh -T git@github.com -o BatchMode=yes -o ConnectTimeout=5
# -> "Hi BonnieNac! You've successfully authenticated..."
```

La clé SSH était bien configurée. On bascule donc le remote de HTTPS vers SSH :

```
git remote set-url origin git@github.com:BonnieNac/agentik_ML_Titanic.git
git push -u origin main
```

Résultat : `* [new branch] main -> main` — les 8 commits locaux sont poussés, le dépôt GitHub n'est plus vide.

À retenir : HTTPS demande un identifiant/jeton à chaque authentification (sauf configuration d'un gestionnaire d'identifiants), alors que SSH s'appuie sur une paire de clés déjà enregistrées côté GitHub — plus adapté à un usage scripté/automatisé.

4. Étape 3 — Récupérer un projet tiers (fork)

Demande : récupérer le squelette externe `malganis35/cookiecutter-astrodata-skeleton` "dans son propre GitHub" — c'est-à-dire en créer une copie liée (un *fork*) sous le compte personnel, pas juste un clone local isolé.

Vérification préalable que l'outil `gh` (GitHub CLI) est bien authentifié :

```
gh auth status
# -> Logged in to github.com account BonnieNac
```

Fork + clone local en une seule commande :

```
gh repo fork malganis35/cookiecutter-astrodata-skeleton --clone=true
```

Résultat : un nouveau dépôt `BonnieNac/cookiecutter-astrodata-skeleton` créé sur GitHub (copie du dépôt original), cloné localement dans `~/cookiecutter-astrodata-skeleton`, avec deux remotes configurés automatiquement :

Remote	Pointe vers	Usage
<code>origin</code>	<code>BonnieNac/cookiecutter-astrodata-skeleton</code>	Le fork personnel — là où on pousse ses propres modifications
<code>upstream</code>	<code>malganis35/cookiecutter-astrodata-skeleton</code>	Le dépôt original — pour récupérer les mises à jour de la source plus tard

5. Étape 4 — Concevoir des visualisations de données propres

5.1 La méthode avant le code : choisir la forme, puis la couleur

Avant d'écrire la moindre ligne de graphique, une méthodologie de dataviz a été appliquée, dans cet ordre précis (la couleur vient **en dernier**, jamais en premier) :

1. **Choisir la forme** selon le "travail" que doit faire la donnée : comparer une magnitude (barres), une tendance (ligne), une répartition (heatmap), une identité (couleur catégorielle)...
2. **Attribuer la couleur selon son rôle** :
 - *séquentiel* (une seule teinte, plus foncé = plus grand) pour comparer une magnitude entre catégories,
 - *catégoriel* (plusieurs teintes) seulement quand les catégories elles-mêmes sont le sujet (ex. "survécu" vs "décédé"),
 - jamais une teinte arc-en-ciel générée au hasard.
3. **Valider la palette** : une palette catégorielle validée à l'avance (testée contre le daltonisme, avec un contraste suffisant) a été réutilisée telle quelle plutôt que d'inventer des couleurs à l'œil :
 - Bleu #2a78d6 — mesure unique (barres mono-teinte)
 - Bleu/Orange #2a78d6 / #eb6834 — paire validée pour 2 catégories (survie)
 - Rampe séquentielle bleu clair→foncé pour la heatmap
4. **Appliquer les règles de lisibilité** : barres fines, angles arrondis, étiquettes de valeur directement sur les barres plutôt qu'une légende séparée quand c'est possible, info-bulles au survol, grille discrète en arrière-plan.
5. **Prévoir une alternative accessible** : une vue "tableau" des données brutes, pour celles et ceux qui ne peuvent pas lire un graphique en couleur.

Pourquoi s'imposer une méthode plutôt que de faire "au feeling" : un graphique mal choisi (ex. un double axe Y, une palette arc-en-ciel, une heatmap sans légende) peut activement induire en erreur. Décider de la forme et de la couleur *avant* d'ouvrir l'éditeur évite de retomber dans ces pièges classiques.

5.2 Explorer l'existant avant d'ajouter du code

Avant d'écrire quoi que ce soit, plusieurs fichiers ont été relus pour ne pas dupliquer ou casser l'existant :

- `app/streamlit_app.py` — l'application actuelle : un explorateur générique de fichiers (CSV/Parquet/Excel) basé sur **Pygwalker**, avec deux onglets ("Aperçu" et "Analyse visuelle")
- `src/titanic_ml/core/features/preprocessing.py` — les colonnes utilisées par le modèle (`Age` , `Fare` , `Pclass` , `Sex` , `Embarked` ...), pour savoir quelles colonnes ont du sens à visualiser
- `src/titanic_ml/core/data_io/loader.py` — la fonction `load_data()` qui télécharge/charge le CSV Titanic, réutilisable telle quelle
- `.streamlit/config.toml` et `app/assets/style.css` — pour respecter le thème visuel déjà en place (clair, palette violet/indigo)

Vérification technique : la bibliothèque de graphiques **Altair** n'était pas listée dans les dépendances directes du projet, mais elle est installée automatiquement comme dépendance indirecte de Streamlit. Elle a été testée avant utilisation :

```
uv run python -c "import altair; print(altair.__version__)"  
# -> 6.3.0
```

Pourquoi l'ajouter explicitement plutôt que de compter sur le hasard : une dépendance "transitive" (installée seulement parce qu'un autre paquet en a besoin) peut disparaître du jour au lendemain si ce paquet change ses propres dépendances. Comme le code importe directement `altair`, elle doit apparaître explicitement dans `pyproject.toml` :

```
uv add altair
```

5.3 Écrire le module de visualisations

Un nouveau fichier `app/viz.py` a été créé, séparé de `streamlit_app.py`, avec :

- Les constantes de couleur de la palette validée, en haut du fichier
- `survival_rate_by(df, colonne, ordre, titre)` — fonction générique qui calcule un taux de survie par catégorie et construit une barre mono-teinte avec étiquette directe et info-bulle, réutilisée pour la classe, le sexe et le port d'embarquement

- `survival_heatmap(df)` — croisement classe × sexe en heatmap séquentielle
- `age_distribution_by_survival(df)` — histogramme superposé de l'âge, coloré par statut de survie
- `prepare_display_frame(df)` — traduit les codes bruts ("male" , "C" , 1 ...) en libellés lisibles en français ("Homme", "Cherbourg", "1ère classe")

5.4 Brancher le nouvel onglet dans l'application

Un troisième onglet "**Visualisations Titanic**" a été ajouté dans `app/streamlit_app.py` , entre les deux onglets existants, contenant :

1. Une ligne de 3 indicateurs clés (nombre de passagers, taux de survie global, âge moyen)
2. Deux graphiques côte à côte : taux de survie par classe / par sexe
3. La heatmap classe × sexe
4. L'histogramme d'âge par statut de survie, et le taux de survie par port d'embarquement
5. Un tableau de données dépliant, en alternative accessible aux graphiques

6. Étape 5 — Le filet de sécurité qualité : lint, typage, tests

Après chaque modification de code, la même série de vérifications automatisées a été relancée — c'est ce que le `Makefile` du projet appelle `make check` :

```
uv run ruff check --fix app/      # style + erreurs évidentes, avec correction automatique
uv run ruff format app/          # mise en forme (indentation, longueur de ligne...)
uv run mypy --explicit-package-bases src/ app/  # vérification stricte des types
uv run pytest tests/ -q          # suite de tests + couverture de code
```

Outil	Ce qu'il vérifie	Problèmes trouvés ici
<code>ruff check</code>	Style, imports inutiles, docstrings manquantes/mal formées	Une ligne trop longue (auto-corrigée), une docstring pas à l'impératif (corrigée à la main)
<code>ruff format</code>	Mise en forme automatique du code	Réindentation, guillemets
<code>mypy --strict</code>	Cohérence des types déclarés	Un type de retour <code>Any</code> implicite et un paramètre <code>None</code> non déclaré comme optionnel — corrigés en précisant <code>str None</code>
<code>pytest</code>	Comportement du code, non-régression	21 tests passés, 98% de couverture (le dossier <code>app/</code> n'est pas soumis au seuil de couverture, par convention du projet)

Pourquoi ces trois couches et pas une seule : chacune attrape une catégorie d'erreur différente. Le style (`ruff`) rend le code lisible par d'autres personnes ; le typage (`mypy`) attrape des erreurs *avant* l'exécution (ex. passer `None` là où une chaîne est attendue) ; les tests (`pytest`) vérifient que le *comportement* du code est correct, ce qu'aucune des deux autres couches ne peut garantir.

7. Étape 6 — Faire tourner l'application pour de vrai

Un principe important a guidé cette étape : **passer le lint et le typage ne prouve pas qu'une interface utilisateur fonctionne**. Un graphique peut être syntaxiquement correct et pourtant illisible (texte qui se chevauche, couleurs qui se confondent). Il faut donc réellement *regarder* le résultat.

7.1 Premier niveau de preuve : rendre les graphiques en image

Plutôt que de se lancer directement dans un test de navigateur complet (plus lourd), chaque graphique Altair a d'abord été exporté en PNG directement en Python, sans navigateur, via la bibliothèque `vl-convert-python` :

```
uv run --with vl-convert-python python - <<'EOF'
import viz
chart = viz.survival_heatmap(df)
chart.save("heatmap.png", scale_factor=2)
EOF
```

Bug trouvé à cette étape : sur la première version de la heatmap, les étiquettes de valeur (ex. "36.9%", "15.7%", "13.5%") se chevauchaient toutes sur une seule ligne, illisibles, à cause d'une largeur de graphique trop étroite. Ce bug n'aurait *pas* été détecté par `ruff` ou `mypy`, qui ne regardent pas le rendu visuel.

Correction : largeur explicite (`width=320`) et hauteur augmentée sur la heatmap, plus l'ajout du symbole "%" dans les étiquettes pour plus de clarté. Nouveau rendu vérifié visuellement : chaque cellule affiche bien sa propre valeur, sans chevauchement.

7.2 Deuxième niveau de preuve : un vrai navigateur headless

Pour vérifier l'application complète (pas juste un graphique isolé), la démarche a été :

1. Démarrer le serveur Streamlit en arrière-plan et attendre qu'il réponde (jamais de `sleep` à l'aveugle, toujours interroger l'état réel) :

```
uv run streamlit run app/streamlit_app.py --server.headless true --server.port 8501 &
timeout 30 bash -c 'until curl -sf http://localhost:8501/_stcore/health; do sleep 1; d
```

2. Chercher un outil de pilotage de navigateur headless déjà installé (`chromium-cli`) — absent dans cet environnement.
3. Installer **Playwright** (bibliothèque Python de pilotage de navigateur) à la volée, sans polluer les dépendances du projet, grâce à `uv run --with` :

```
uv run --with playwright python -m playwright install chromium
```

4. Piloter le navigateur : ouvrir la page, cliquer sur l'onglet "Visualisations Titanic", attendre le rendu, prendre une capture d'écran, et vérifier la console du navigateur (aucune erreur JavaScript).

7.3 Un obstacle système résolu sans droits root

Erreur rencontrée : le navigateur Chromium refusait de démarrer : `error while loading shared libraries: libasound.so.2: cannot open shared object file` (une bibliothèque audio manquante). La solution habituelle, `playwright install --with-deps`, demande les droits administrateur (`sudo`) — indisponibles ici de façon interactive.

Solution trouvée sans avoir besoin des droits root : télécharger uniquement le paquet Debian contenant cette bibliothèque (sans l'installer sur le système), l'extraire dans un dossier temporaire, puis dire au système où la trouver via une variable d'environnement :

```
apt-get download libasound2t64          # télécharge le .deb, sans droits admin
dpkg-deb -x libasound2t64_*.deb /tmp/localdeps # extrait son contenu localement
LD_LIBRARY_PATH=/tmp/localdeps/usr/lib/x86_64-linux-gnu uv run --with playwright python .
```

Résultat : le navigateur headless démarre, la page se charge, les captures d'écran confirment un rendu propre du tableau de bord, sans erreur de console.

Pourquoi cette étape compte : face à un obstacle technique, la première question n'est pas "comment forcer le passage" (ici, demander `sudo` à l'utilisatrice pour une simple vérification) mais "existe-t-il un chemin qui n'exige pas plus de droits que nécessaire ?". Télécharger et isoler une seule bibliothèque évite de modifier le système pour une opération ponctuelle de vérification.

8. Étape 7 — Livrer la fonctionnalité

Une fois le code vérifié (lint + typage + tests + rendu réel), il a été committé :

```
git add app/viz.py app/streamlit_app.py pyproject.toml uv.lock
git commit -m "feat: ajoute un tableau de bord de visualisations claires du dataset Titar"
```

Pourquoi demander confirmation avant de pousser sur GitHub :

committer est une action locale et réversible ; pousser (`git push`) rend le changement visible sur un dépôt partagé. Une bonne pratique consiste à toujours redemander confirmation avant une action visible par d'autres, même si un push précédent avait déjà été autorisé dans la conversation — l'autorisation ne se reporte pas automatiquement d'une action à l'autre.

Après confirmation ("oui") :

```
git push origin main
```

9. Étape 8 — Un vrai bug remonté par l'utilisatrice

En essayant l'application elle-même, l'utilisatrice a obtenu une erreur bloquant toute l'application :

```
ImportError: cannot import name 'make_url_path_regex'  
  from 'streamlit.web.server.server_util'  
(déclenché par : from pygwalker.api.streamlit import StreamlitRenderer)
```

Question posée : "c'est dû au fait que je sois sur WSL ?"

Réponse et raisonnement : non — ce n'est pas un problème d'environnement (WSL, Linux, Mac...), mais une incompatibilité pure entre deux versions de bibliothèques Python. `pygwalker` (sa dernière version publiée) importe une fonction interne de Streamlit qui a été supprimée dans les versions récentes de Streamlit. Comme `pygwalker` ne précise pas de version maximale de Streamlit qu'il supporte, le gestionnaire de paquets (`uv`) avait installé la toute dernière version de Streamlit — incompatible. Ce même plantage se serait produit sur n'importe quel système d'exploitation.

Premier correctif (limiter les dégâts) : plutôt que de se précipiter sur une correction incertaine des versions, l'import cassé a été isolé pour qu'il ne fasse plus tomber *toute* l'application :

- L'import de `pygwalker` a été déplacé du haut du fichier (exécuté au démarrage, pour tous les onglets) vers l'intérieur de la fonction qui en a réellement besoin (seulement quand on ouvre l'onglet "Analyse visuelle")
- Cet import est entouré d'un `try/except ImportError` qui affiche un message d'erreur clair à l'utilisatrice plutôt que de laisser Python remonter une trace technique brute

Pourquoi isoler le problème avant de le résoudre : avec l'import en tête de fichier, même les onglets "Aperçu" et "Visualisations Titanic" — qui n'ont rien à voir avec `pygwalker` — étaient inutilisables. Isoler l'erreur permet de rendre le reste de l'application utilisable immédiatement, pendant que la vraie cause est encore en cours d'investigation.

10. Étape 9 — Corriger le bug pour de bon

Le correctif précédent masquait le symptôme sans le résoudre : l'utilisatrice a de nouveau signalé le message d'erreur (attendu, mais pas satisfaisant en soi). Il fallait trouver *quelle* version de Streamlit reste compatible avec `pywalker`.

Bissection de versions

Plutôt que d'essayer une version au hasard, une recherche par dichotomie ("bissection") a été menée : installer différentes versions de Streamlit dans un environnement jetable (sans toucher au projet), et tester si l'import fonctionne :

```
uv run --with streamlit==1.50.0 python -c \  
    "from streamlit.web.server.server_util import make_url_path_regex"
```

Version Streamlit testée	Résultat
1.50.0 / 1.55.0 / 1.56.0	☐ import réussi
1.57.0 / 1.58.0 / 1.60.0 / 1.62.0 / 1.63.0 / 1.64.0	✕ <code>ImportError</code>

Frontière identifiée : Streamlit 1.56.0 est la dernière version compatible ; l'incompatibilité commence exactement à la version 1.57.0.

Pourquoi tester méthodiquement plutôt que deviner : une bissection (tester un point médian, puis resserrer l'intervalle) trouve la frontière exacte en quelques essais, au lieu de "tâtonner" une version arbitrairement plus ancienne — qui aurait pu être une rétrogradation inutilement importante (avec son lot de nouveaux risques) ou encore insuffisante.

Appliquer et vérifier le correctif

```
uv add "streamlit<1.57"
```

Puis, comme à chaque changement, l'intégralité du filet de sécurité a été rejouée :

```
uv run pytest tests/ -q          # 21 passed, couverture 98%  
uv run ruff check app/ src/ tests/ # OK  
uv run mypy --explicit-package-bases src/ app/ # OK
```

Et surtout, la vérification visuelle en navigateur headless a été refaite (voir §7.2) pour confirmer que l'explorateur Pygwalker ne se contente pas de ne plus planter, mais fonctionne réellement : la capture d'écran a montré l'interface de glisser-déposer des colonnes pleinement opérationnelle.

Pourquoi revérifier visuellement après un simple changement de version : l'absence d'erreur ne prouve pas qu'une fonctionnalité marche — elle peut simplement échouer plus silencieusement. Le critère de succès retenu ici était "l'interface s'affiche et répond au clic", pas seulement "aucune exception n'est levée".

Le correctif a ensuite été committé et poussé :

```
git commit -m "fix: fige streamlit<1.57 pour rester compatible avec pygwalker"
git push origin main
```

11. Étape 10 — Clôturer proprement

- Sur nouvelle demande de push, vérification qu'il n'y avait *rien* de nouveau à committer ou pousser (`git status` propre, `main` local strictement identique à `origin/main`) plutôt que de pousser "pour rien"
- Sur demande d'arrêt, le serveur Streamlit lancé en arrière-plan a été arrêté proprement en libérant le port utilisé :

```
lsof -ti:8501 -sTCP:LISTEN | xargs kill
```

- Les fichiers temporaires de vérification (captures d'écran, PNG de test) créés dans un dossier de travail (*scratchpad*) dédié — jamais dans le dépôt du projet — ont été nettoyés une fois leur rôle de preuve rempli.

12. Ce qu'il faut retenir

Au-delà du projet Titanic lui-même, les principes appliqués tout au long de ce travail sont transférables à n'importe quel projet de data science :

1. **Lire avant d'écrire.** Comprendre les conventions et l'architecture existantes avant d'ajouter du code, pour ne pas les contredire.
2. **Séparer les préoccupations.** Le nouveau code de visualisation (`viz.py`) a été isolé de l'application (`streamlit_app.py`) plutôt que tout mélanger.
3. **Décider la forme et la couleur d'un graphique avant de coder**, avec une palette et des règles de lisibilité validées à l'avance plutôt que choisies à l'instinct.
4. **Un filet de sécurité à trois niveaux** (style, typage, tests) rattrape des catégories d'erreurs différentes et complémentaires.
5. **Le typage et les tests ne suffisent pas pour une interface** : il faut la faire tourner et la regarder — idéalement dans un vrai navigateur, pas seulement "ça s'importe sans erreur".
6. **Face à un obstacle système, chercher la solution la moins invasive** (extraire une bibliothèque manquante plutôt que demander des droits administrateur pour une simple vérification).
7. **Diagnostiquer par élimination méthodique** (bissection de versions) plutôt que par essais au hasard, surtout face à des messages d'erreur qui pointent vers du code interne d'une bibliothèque.
8. **Isoler un problème avant de le résoudre** : rendre le reste du système utilisable pendant l'investigation, plutôt que de tout bloquer en attendant le correctif définitif.
9. **Committer avec des messages structurés** (conventional commits), et demander confirmation avant toute action visible par d'autres (un `git push`), même après une autorisation précédente dans la même conversation.
10. **Nettoyer derrière soi** : fichiers temporaires, serveurs de test, une fois leur rôle rempli.

Fin du document.